

LESSON 2

LINKED LIST



CCE105
DATA
STRUCTURES

TOPICS



LINKED

- LIST
- INSERTION
- DELETION
- DISPLAYING
- SEARCHING

TOPICS



2

DOUBLE-ENDED LINKED LIST

- INSERTION
- DELETION
- DISPLAYING

TOPICS



DOUBLY LINKED LIST

- INSERTION
- DELETION
- DISPLAYING

LINKED LIST



Often in programming we are required to systematically store some type of information. A prime example of this is to use arrays, but when you don't know the amount of information to be stored we need a dynamic data structure.

One option for us is to use a linked list.

A **linked list** works by creating a collection of objects (nodes) which both carry the data we want to store and a reference to the next node in the list.

LINKED LIST



A linked data structure consists of capsules of data known as **nodes** that are connected via **links**.

Links can be viewed as arrows and thought of as one way passages from one node to another.

In Java, nodes are realized as objects of a node class. The data in a node is stored via instance variables.

The links are realized as references:

- A reference is a memory address, and is stored in a variable of a class type
- Therefore, a link is an instance variable of the node class type itself

LINKED LIST



A linked list consists of a single chain of nodes, each connected to the next by a link

- The first node is called the **head** node
- The last node serves as a kind of **end** marker



LINKED LIST



The order of the nodes is determined by the address, called the **link**, stored in each node

- Every node (except the last node) contains the address of the next node
- Components of a node
 - **Data:** stores the relevant information
 - **Link:** stores the address of the next node

LINKED LIST



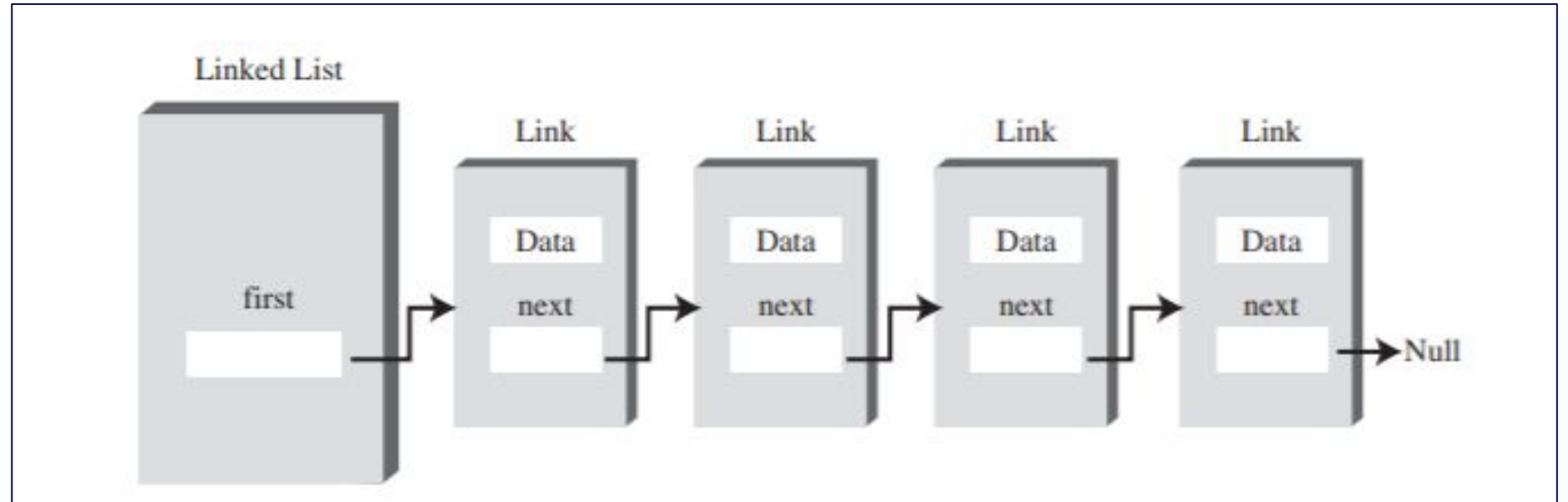
Class Node

- Represents nodes on a list
- It has two instance variables
 - **info** (of type int, but it can be any other type)
 - **link** (of type Node)

LINKED LIST



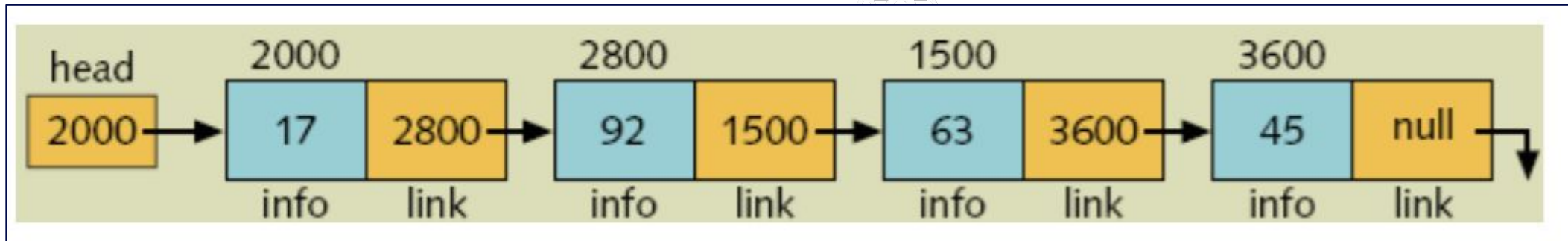
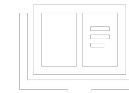
1



LINKED LIST



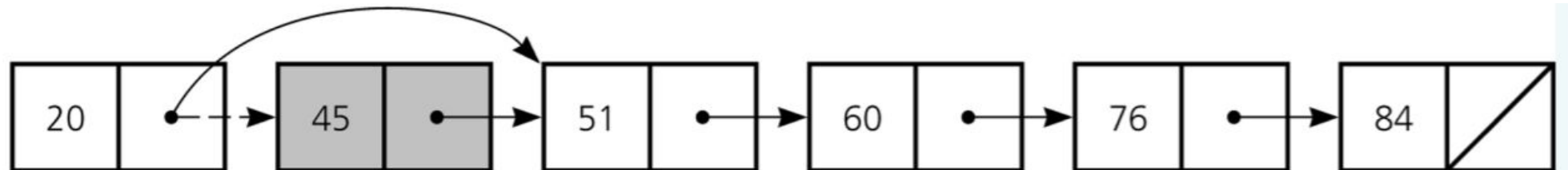
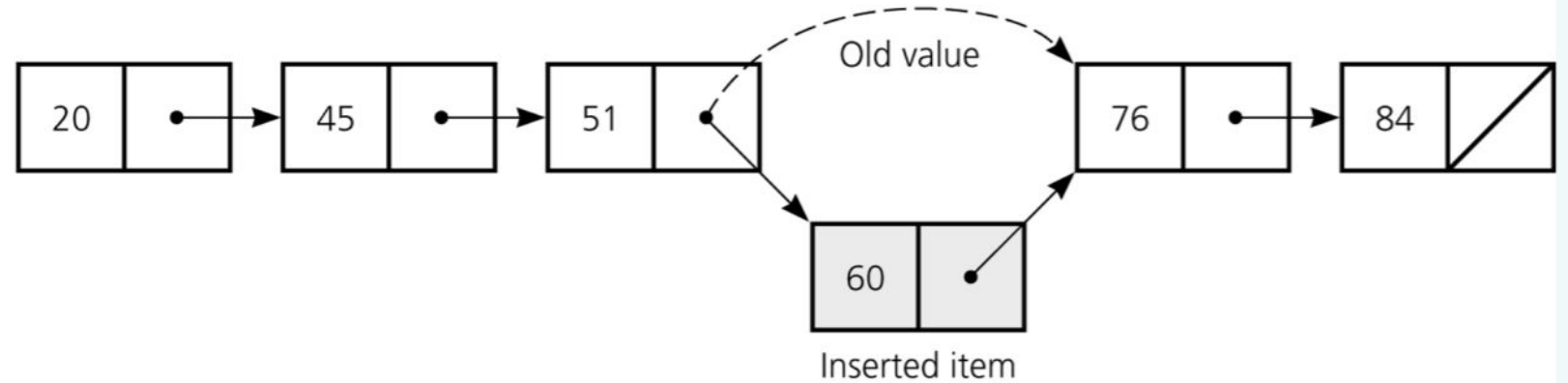
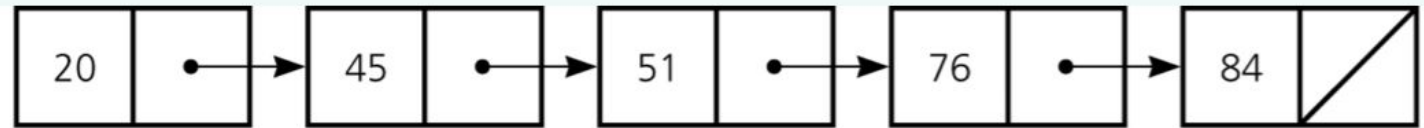
2

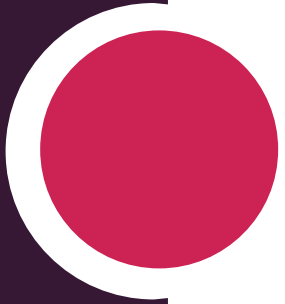


LINKED LIST



3

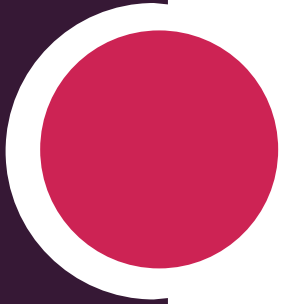




LINKED LIST



```
class Link
{
    public int iData;           // data item
    public double dData;       // data item
    public Link next;          // next link in list
    // -----
    public Link(int id, double dd) // constructor
    {
        iData = id;           // initialize data
        dData = dd;           // ('next' is automatically
                               // set to null)
    }
    // -----
    public void displayLink()    // display myself
    {
        System.out.print("{ " + iData + ", " + dData + " } ");
    }
} // end class Link
```



LINKED LIST



```
class LinkList
{
    private Link first;           // ref to first link on list

    // -----
    public void LinkList()        // constructor
    {
        first = null;            // no items on list yet
    }

    // -----
    public boolean isEmpty()       // true if list is empty
    {
        return (first==null);
    }

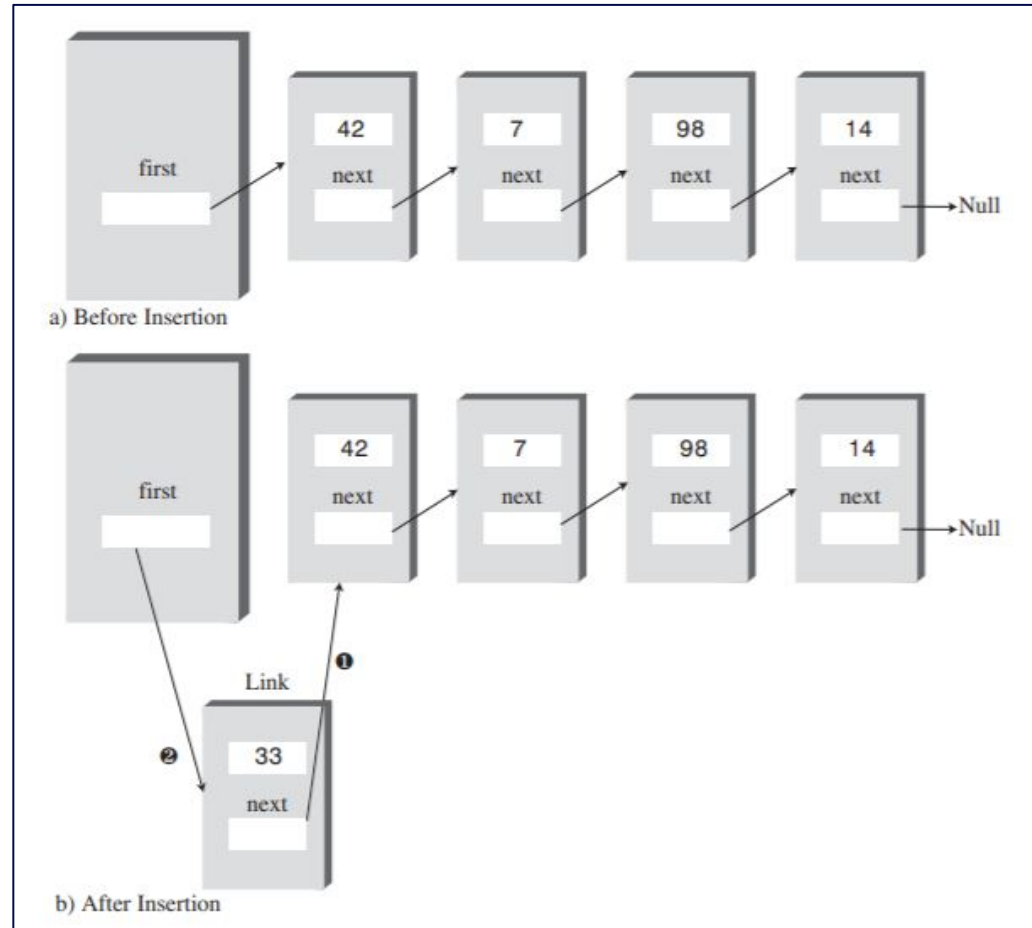
    // -----
    // ... other methods go here
}
```

The **Java String class isEmpty()** method checks if the input string is empty or not. Note that here empty means the number of characters contained in a string is zero. **Returns** true if length is 0 otherwise false.

LINKED LIST

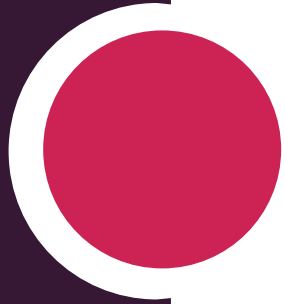


1



The insertFirst() Method

- The insertFirst() method of LinkedList inserts a new link at the beginning of the list.



LINKED LIST



Insert the following: 14 98 7 42

```
// insert at start of list
public void insertFirst(int id, double dd)
{
    // make new link
    Link newLink = new Link(id, dd);
    newLink.next = first;    // newLink --> old first
    first = newLink;        // first --> newLink
}
```

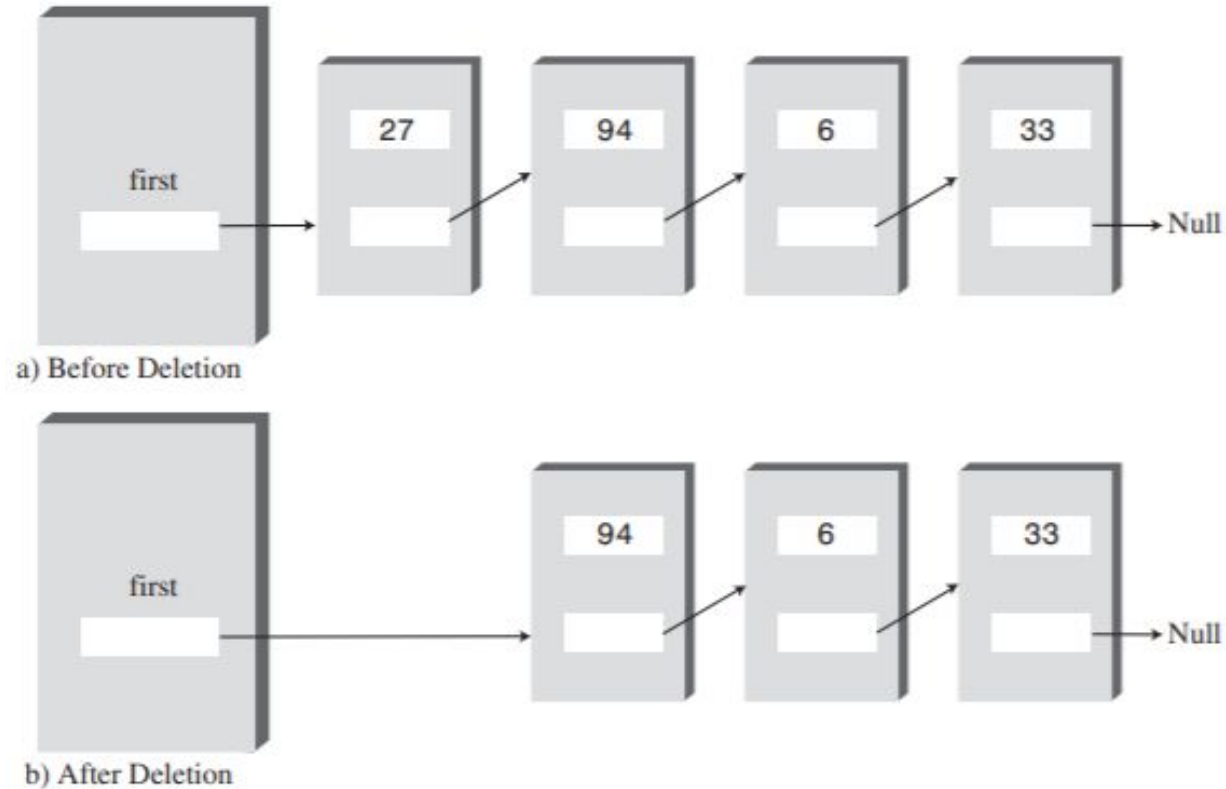
The **insertFirst()** Method

- The **insertFirst()** method of **LinkedList** inserts a new link at the beginning of the list.

LINKED LIST

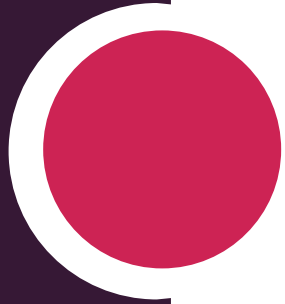


2



The `deleteFirst()` Method

- It disconnects the first link by rerouting first to point to the second link. This second link is found by looking at the next field in the first link



LINKED LIST



Let us try to delete using deleteFirst():
(assuming we have values for dd)



```
public Link deleteFirst()    // delete first item
{                             // (assumes list not empty)
    Link temp = first;       // save reference to link
    first = first.next;      // delete it: first-->old next
    return temp;             // return deleted link
}
```

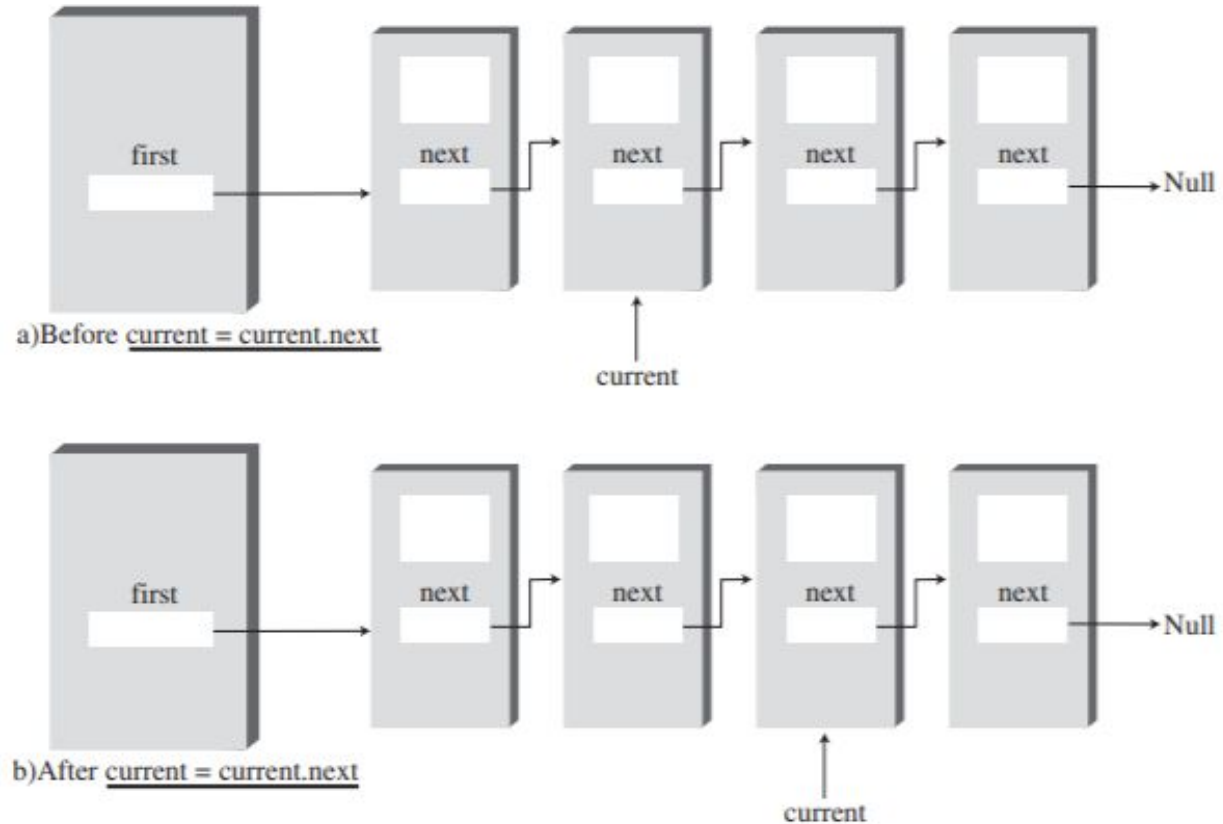
The deleteFirst() Method

- It disconnects the first link by rerouting first to point to the second link. This second link is found by looking at the next field in the first link

LINKED LIST

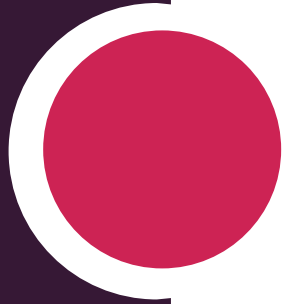


3



The `displayList()` Method

- To display the list, you start at first and follow the chain of references from link to link. A variable current points to (or technically refers to) each link in turn. It starts off pointing to first, which holds a reference to the first link.

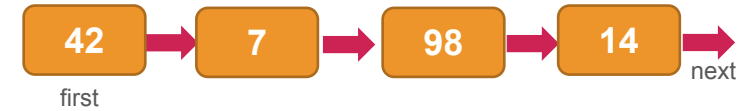


LINKED LIST



The `displayList()` Method

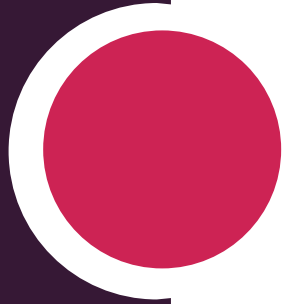
Displaying nodes using `displayList()`
Assuming we have value for `dData`



```

public void displayList()
{
    System.out.print("List (first-->last): ");
    Link current = first;      // start at beginning of list
    while(current != null)     // until end of list,
    {
        current.displayLink(); // print data
        current = current.next; // move to next link
    }
    System.out.println("");
}

public void displayLink()      // display ourself
{
    System.out.print("{ " + iData + ", " + dData + " } ");
}
  
```



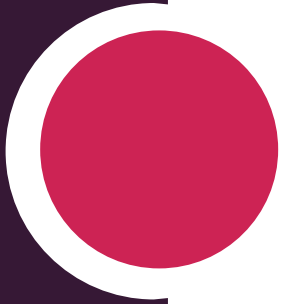
LINKED LIST



The Main Class

```
class LinkListApp
{
    public static void main(String[] args)
    {
        LinkList theList = new LinkList(); // make new list
        theList.insertFirst(14, 2.99); // insert four items
        theList.insertFirst(98, 4.99);
        theList.insertFirst(7, 6.99);
        theList.insertFirst(42, 8.99);
        theList.displayList(); // display list
        while( !theList.isEmpty() ) // until it's empty,
        {
            Link aLink = theList.deleteFirst(); // delete link
            System.out.print("Deleted "); // display it
            aLink.displayLink();
            System.out.println("");
        }
        theList.displayList(); // display list

    } // end main()
}
// end class LinkListApp
```



LINKED LIST



OUTPUT

```
List (first-->last): {42, 8.99} {7, 6.99} {98, 4.99} {14, 2.99}
Deleted {42, 8.99}
Deleted {7, 6.99}
Deleted {98, 4.99}
Deleted {14, 2.99}
List (first-->last):
```

LINKED LIST



Finding and Deleting Specified Links

```
class Link2
{
    public int iData; // data item (key)
    public double dData; // data item
    public Link2 next; // next link in list
    // -----
    public Link2(int id, double dd) // constructor
    {
        iData = id;
        dData = dd;
    }
    // -----
    public void displayLink() // display myself
    {
        System.out.print("{ " + iData + ", " + dData + " } ");
    }
} // end class Link
```


LINKED LIST



Finding and Deleting Specified Links

```
class LinkedList
{
private Link2 first; // ref to first link on list
// -----
public LinkedList() // constructor
{
first = null; // no links on list yet
}
// -----
public void insertFirst(int id, double dd)
{ // make new link
Link2 newLink = new Link2(id, dd);
newLink.next = first; // it points to old first link
first = newLink; // now first points to this
}
// -----
```


LINKED LIST



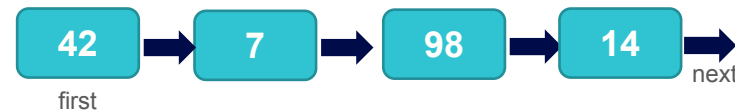
Finding and Deleting Specified Links

- Using find() method

```
public Link2 find(int key) // find link with given key
{ // (assumes non-empty list)
  Link2 current = first; // start at 'first'
  while(current.iData != key) // while no match,
  {
    if(current.next == null) // if end of list,
      return null; // didn't find it
    else // not end of list,
      current = current.next; // go to next link
  }
  return current; // found it
}
// -----
```



Displaying nodes using displayList()
Assuming we have value for dData



LINKED LIST

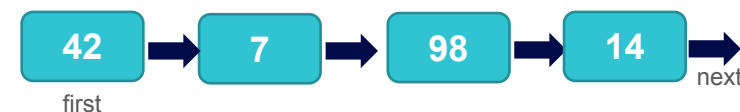


Finding and Deleting Specified Links

- Using delete() method
 - Deleting 98

```
public Link2 delete(int key) // delete link with given key
{ // (assumes non-empty list)
    Link2 current = first; // search for link
    Link2 previous = first;
    while(current.iData != key)
    {
        if(current.next == null)
            return null; // didn't find it
        else
        {
            previous = current; // go to next link
            current = current.next;
        }
    } // found it
    if(current == first) // if first link,
        first = first.next; // change first
    else // otherwise,
        previous.next = current.next; // bypass it
    return current;
}
// -----
```

Displaying nodes using displayList()
Assuming we have value for dData



LINKED LIST



Finding and Deleting Specified Links

```
public void displayList()      // display the list
{
    System.out.print("List (first-->last): ");
    Link current = first;      // start at beginning of list
    while(current != null)     // until end of list,
    {
        current.displayLink(); // print data
        current = current.next; // move to next link
    }
    System.out.println("");
}

// -----
} // end class LinkedList
// -----
```

LINKED LIST



Finding and Deleting Specified Links

```
class LinkList2App {
    public static void main(String[] args){
        LinkList theList = new LinkList(); // make list
        theList.insertFirst(14, 2.99); // insert 4 items
        theList.insertFirst(98, 4.99);
        theList.insertFirst(7, 6.99);
        theList.insertFirst(42, 8.99);

        theList.displayList(); // display list
        Link2 f = theList.find(98); // find item

        if( f != null)
            System.out.println("Found link with key " + f.iData);
        else
            System.out.println("Can't find link");

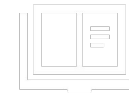
        Link2 d = theList.delete(98); // delete item
        if( d != null )
            System.out.println("Deleted link with key " + d.iData);
        else
            System.out.println("Can't delete link");

        theList.displayList(); // display list
    } // end main()
} // end class LinkList2App
```

LINKED LIST



OUTPUT

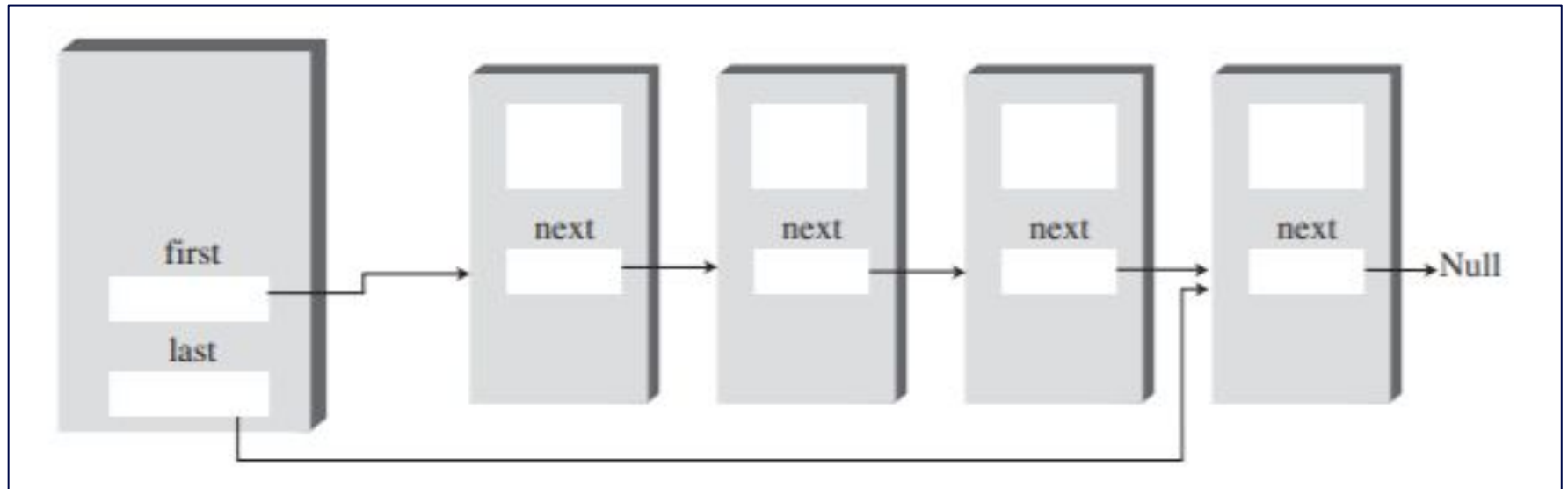


```
List (first-->last): {42, 8.99} {7, 6.99} {98, 4.99} {14, 2.99}  
Found link with key 98  
Deleted link with key 98  
List (first-->last): {42, 8.99} {7, 6.99} {14, 2.99}
```

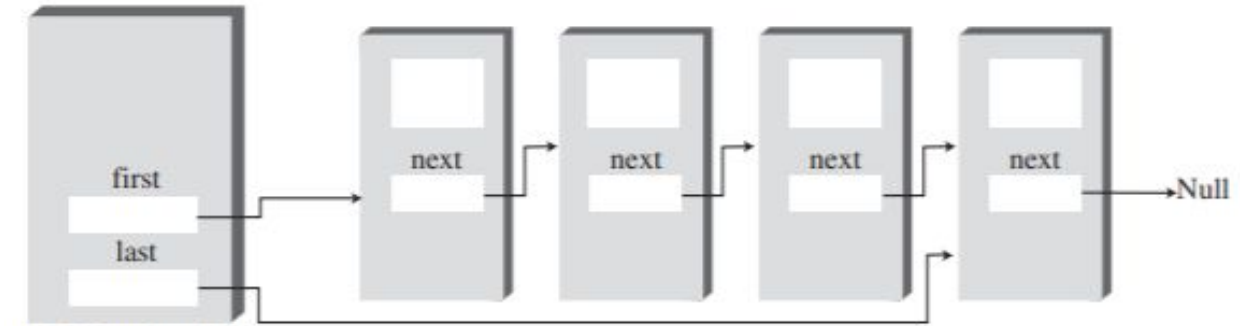
DOUBLE-ENDED LINKED LIST



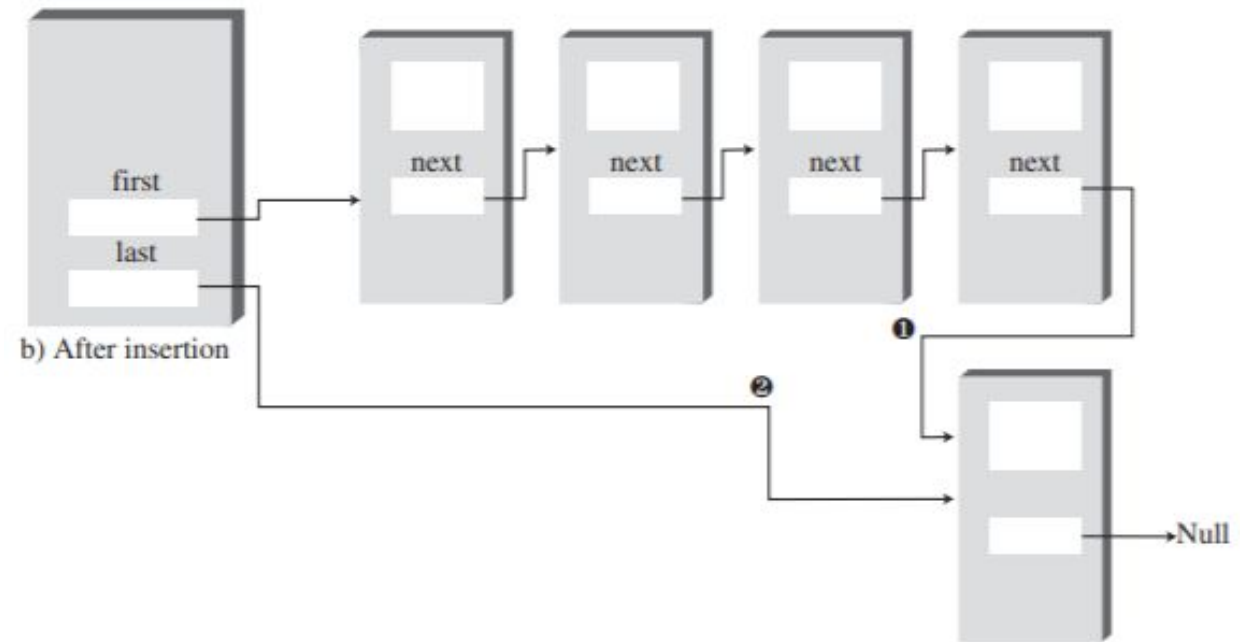
A double-ended linked list is similar to an ordinary linked list, but it has one additional feature: a reference to the last link as well as to the first.



DOUBLE-ENDED LINKED LIST



a) Before insertion



b) After insertion


```
class Link
{
    public long dData;           // data item
    public Link next;           // next link in list
// -----
    public Link(long d)         // constructor
    { dData = d; }
// -----
    public void displayLink()    // display this link
    { System.out.print(dData + " "); }
// -----
} // end class Link
////////////////////////////////////
```


DOUBLE-ENDED LINKED LIST



```
class FirstLastList
{
    private Link first;           // ref to first link
    private Link last;           // ref to last link
// -----
    public FirstLastList()       // constructor
    {
        first = null;           // no links on list yet
        last = null;
    }
// -----
    public boolean isEmpty()      // true if no links
    { return first==null; }
// -----
```

*The **Java String** class **isEmpty()** method checks if the input string is empty or not. Note that here empty means the number of characters contained in a string is zero. Returns true if length is 0 otherwise false.*

DOUBLE-ENDED LINKED LIST



- Using insertFirst() method

```
public void insertFirst(long dd) // insert at front of list
{
    Link newLink = new Link(dd); // make new link

    if( isEmpty() )                // if empty list,
        last = newLink;           // newLink <-- last
    newLink.next = first;          // newLink --> old first
    first = newLink;              // first --> newLink
}

// -----
```

Let us try to insert the following values:

22 44 66

DOUBLE-ENDED LINKED LIST



- Using insertLast() method

```
public void insertLast(long dd)    // insert at end of list
{
    Link newLink = new Link(dd);    // make new link
    if( isEmpty() )                // if empty list,
        first = newLink;            // first --> newLink
    else
        last.next = newLink;        // old last --> newLink
    last = newLink;                // newLink <-- last
}

// -----
```

Let us try to insert the following values:

11 33 55

[illegible]

DOUBLE-ENDED LINKED LIST



- Using deleteFirst() method

```
public long deleteFirst()           // delete first link
{                                   // (assumes non-empty list)
    long temp = first.dData;
    if(first.next == null)          // if only one item
        last = null;               // null <-- last
    first = first.next;             // first --> old next
    return temp;
}
```



A decorative graphic in the top right corner. It features four overlapping diamonds in red, orange, purple, and teal. To the right of the diamonds, the word "DOUB" is written in a dark blue, bold, sans-serif font.



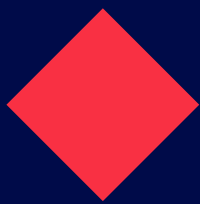
////////////////////////////////////

DOUBLE-ENDED LINKED LIST



OUTPUT

```
List (first-->last): 66 44 22 11 33 55  
List (first-->last): 22 11 33 55
```



THANKS
THANKS
THANKS

